SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4311

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Code of Conduct:

I ___P.Ranjithkumar (192411119)_____ (Name / Reg No) certify that this submission is my originalwork and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:P.Ranjithkumar

1. HTTP Response Analysis

The server response contains useful status and metadata fields, but the declared body length is invalid. HTTP response fields must follow the expected syntax so the browser can correctly interpret the message.

1.1 Identify incorrect or invalid fields

The main error is Content-Length: -150. Content-Length represents the number of octets (bytes) in the message body and cannot be negative. The Date field also contains a mismatch: 1 April 2026 is a Wednesday, not Monday.

- HTTP/1.1 200 OK — a valid status line indicating that the request was successfully handled.
- Date — indicates when the response was generated; its weekday should match the calendar date.
- Content-Type: text/html — valid when the response body contains HTML.
- Content-Length: -150 — invalid because a message body cannot have a negative byte length.
- Connection: keep-alive — valid in HTTP/1.1, where persistent connections are normally the default.

1.2 Analyze the Content-Length issue and its impact

Content-Length tells the client how many bytes are present in the response body. A negative value violates the field's expected format and cannot describe a real body. A strict implementation may reject the malformed response or treat it as a protocol error. Incorrect message framing can also affect how the client determines the end of a response and whether a connection can be reused safely.

The field must contain a non-negative decimal integer that matches the actual number of body bytes. If the response body is exactly 150 bytes, the field should be Content-Length: 150. If the length is not known in advance, the server should use appropriate HTTP framing rather than inventing a value.

1.3 Determine whether the status code matches response correctness

The status code 200 OK indicates that the request was successfully handled, but it does not guarantee that every response header is valid. In this example, the status line may indicate success while the malformed Content-Length makes the overall message defective. Status meaning and message-format correctness are separate issues.

1.4 Suggest corrections

A corrected example, assuming the body is exactly 150 bytes, is:

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

The value 150 is correct only if the actual response body contains 150 bytes. In a real application, the server or framework should calculate this value automatically.

2. DOM Structure Analysis

The supplied tree contains a paragraph element without an explicit closing tag. Browsers recover from many markup errors, but the source should be corrected so the intended document structure is clear.

2.1 Identify missing or improperly closed elements

```
<body>
  <div>
    <p>HelloWorld
  </div>
</body>
```

The paragraph starts with but does not explicitly end with before the div closes. A clearer version is:

```
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
```

2.2 Analyze how the browser interprets the structure

The HTML parser applies error-recovery rules while constructing the DOM. In this case, it will commonly infer the end of the paragraph when it reaches the closing div, then continue parsing. This recovery helps the page display, but developers should not depend on implicit correction for predictable results.

2.3 Determine the impact on rendering and layout

- The text may still appear because browsers attempt to recover from common HTML errors.
- CSS rules based on element relationships may behave differently from the author's intention.
- JavaScript that searches for or modifies nodes can be harder to maintain if the source structure is ambiguous.
- Accessibility tools and validators may report the malformed source; more complex cases can cause confusing structure or behaviour.

The visible impact may be small in this example, but similar omissions in a larger page can cause confusing layout and DOM behaviour.

2.4 Suggest corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Test Page</title>
</head>
<body>
   <main>
      <div>
         <p>Hello World</p>
      </div>
   </main>
</body>
</html>
```

This version closes the paragraph, uses the HTML5 declaration, specifies the document language and character encoding, and includes viewport metadata for mobile devices.

3. GET vs POST in Login System

GET and POST have different purposes and different ways of carrying form data. Neither method alone encrypts data; HTTPS provides protection for data while it travels over the network.

3.1 Interpret differences in data transmission

With GET, form values are commonly appended to the URL as query parameters. With POST, form values are normally carried in the request body. For example:

```
GET /login?user=admin&pass=1234 HTTP/1.1

POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded


user=admin&pass=1234
```

GET is suitable for retrieving or searching information when parameters may be shareable. POST is generally more appropriate for login credentials because it avoids placing them directly in the URL.

3.2 Which method is more suitable for login and why?

POST is more suitable for login because credentials are placed in the request body rather than the URL. However, POST alone is not fully secure: without HTTPS, request data may still be observed by someone monitoring the connection. Secure authentication requires POST over HTTPS and appropriate server-side protections.

3.3 Identify security risks in using GET

- Credentials may appear in browser history and in copied or bookmarked URLs.
- URLs can be recorded in server, proxy, analytics, or monitoring logs.
- Sensitive query parameters may be exposed in screenshots or shared links.
- Depending on browser and policy settings, URLs can be disclosed in some referrer or diagnostic contexts.
- Secrets in URLs can be retained longer than intended.

3.4 Best practices for secure login

- Serve the login page and authentication endpoint over HTTPS.
- Use POST to submit usernames and passwords.
- Validate credentials on the server; browser validation alone is not a security control.
- Store passwords using a modern password-hashing algorithm, never as plain text.
- Use secure session handling and suitable cookie attributes, including Secure and HttpOnly; choose SameSite appropriately.
- Apply rate limiting and monitoring to reduce automated password-guessing attacks.
- Consider multi-factor authentication where appropriate.
- Do not place passwords in URLs, logs, or error messages.

4. HTML Code Inspection

The sample page contains omitted closing tags and an image element without alternative text. Browsers may still display the page, but the markup should be corrected for clarity and accessibility.

4.1 Identify missing closing tags

```
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
```

The h1 heading is not explicitly closed with </h1>, and the paragraph is not explicitly closed with </p>. The img element is a void element in HTML, so it does not require a separate closing tag.

4.2 Analyze issues with the img tag

The image has a src attribute but no alt attribute. Alternative text should describe the meaning of an informative image. If the image is purely decorative, use alt="" so assistive technology can ignore it. The file must also exist at the specified path and be accessible. Explicit width and height can help reduce layout shifts while an image loads.

4.3 Determine how browsers handle these errors

Browsers attempt to recover from missing end tags and may implicitly close the heading or paragraph while parsing. The page may look mostly normal, but malformed markup can produce unexpected DOM relationships and make styling, scripting, and accessibility less predictable. If the image cannot load, the browser may show a broken-image indicator; useful alternative text provides a fallback.

4.4 Suggest a corrected version

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Sample Page</title>
</head>
<body>
   <main>
      <h1>Welcome</h1>
      <p>This is a paragraph describing the page content.</p>
      <img src="image.jpg"
          alt="Students participating in a college event"
          width="600"
          height="350">
   </main>
</body>
</html>
```

The sample alt text assumes the image actually shows students at an event. It must be changed to accurately describe the real image.

4.5 Additional improvements

- Use a descriptive page title and a logical heading hierarchy.
- Write accurate alt text for meaningful images and empty alt text for decorative images.
- Check the image path and confirm the file loads.
- Use responsive styling such as max-width: 100%; height: auto; so the image fits small screens.
- Validate the page and test keyboard and screen-reader accessibility.

5. Browser-Server Communication Flow

Opening a website involves coordinated operations between the user's device, the network, and the web server. Understanding each stage helps identify delays and improve page-loading speed.

5.1 Interpret each step in the communication flow

1. **URL entry:** The user enters a web address or selects a link. The browser identifies the scheme, host, path, and query parameters.
2. **DNS lookup:** If no suitable cached result exists, the browser or operating system resolves the host name to an IP address using DNS.
3. **Connection and HTTP request:** The browser establishes a network connection. For HTTPS, TLS negotiation protects the connection. The browser then sends an HTTP request with a method, path, headers, and sometimes a body.
4. **Server processing:** The server routes the request, executes application logic, accesses data or other services when required, and prepares a response.
5. **HTTP response:** The server returns a status code, headers, and possibly a body containing HTML, JSON, images, stylesheets, scripts, or a redirect.
6. **Browser rendering:** The browser parses HTML into the DOM, processes CSS, executes applicable JavaScript, fetches referenced resources, and paints the page.

5.2 Identify where delays can occur

- DNS lookup may take longer when the answer is not cached or the resolver is slow.
- Network latency, weak connectivity, or packet loss can delay connection setup and data transfer.
- TLS negotiation adds connection work for HTTPS, although connection reuse can reduce repeated overhead.
- Slow application logic, database queries, or an overloaded server can delay the response.
- Large images, scripts, and uncompressed resources take longer to download.
- Render-blocking stylesheets, scripts, or a very complex DOM can delay the first useful display.

5.3 Analyze the role of DNS

DNS, the Domain Name System, translates a human-readable host name such as www.example.com into an IP address that a device can use to contact the server. DNS caching avoids repeating lookups while a cached record remains valid. DNS helps locate the server, but it does not itself deliver the webpage or guarantee that the server is available.

5.4 Suggest performance optimizations

- Use reliable DNS hosting and sensible caching policies.
- Use a content delivery network (CDN) for static files when users are geographically distributed.
- Enable compression such as Brotli or gzip for suitable text resources.
- Resize and optimize images; use modern image formats where appropriate.
- Remove unnecessary JavaScript and CSS, and defer non-critical scripts.
- Set cache-control headers for reusable static resources.
- Optimize server code and database queries, and monitor response times.
- Use persistent connections and modern HTTP versions supported by the deployment.
- Measure actual performance with browser developer tools and real-user monitoring before choosing an optimization.

Conclusion

These five cases cover HTTP response-header correctness, DOM construction, authentication methods, HTML validation, and browser-server communication. Correct protocol metadata helps clients interpret responses; valid HTML makes the DOM more predictable; POST over HTTPS is appropriate for login credentials; accessible image markup improves usability; and performance monitoring helps locate delays. Applying these principles supports web applications that are more reliable, accessible, secure, and efficient.

RUBRICS FOR CALCULATION:

Criteria	Weighta ge	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25% (23)	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25% (23)	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20% (18)	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20% (18)	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10% (8)	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand